> **DEFINITION** An operation is ***idempotent*** if performing it multiple times has the same effect as performing it just once. Idempotent operations are important for reliability because if a request fails because of a network error the client can safely retry it.

The HTTP specification requires the read-only methods GET, HEAD, and OPTIONS, along with PUT and DELETE requests, to all be idempotent. Only the POST and PATCH methods are not generally idempotent.

> **WARNING** Even if you stick to PUT requests instead of POST, this doesn't mean that your requests are always safe from replay.

The problem is that the definition of idempotency says nothing about what happens if another request occurs in between the original request and the replay. For example, suppose you send a PUT request updating a page on a website, but you lose your network connection and do not know if the request succeeded or not. Because the request is idempotent, you send it again. Unknown to you, one of your colleagues in the meantime sent a DELETE request because the document contained sensitive information that shouldn't have been published. Your replayed PUT request arrives afterwards, and the document is resurrected, sensitive data and all. An attacker can replay requests to restore an old version of a resource, even though all the operations were individually idempotent.

Thankfully there are several mechanisms you can use to ensure that no other request has occurred in the meantime.